☑ Project Summary: RRCC Space Grant Robotics Team

🚀 Objective

Develop an autonomous rover modeled after NASA Mars rovers, for the **NASA Colorado Space Grant Consortium Robotics Challenge**, as an **educational, long-term, team-based engineering platform** for Red Rocks Community College (RRCC) students.

🧠 Project Philosophy

- **Designed to evolve** over time with contributions from successive student cohorts.
- Emphasizes **modular C++/Arduino-based code architecture** to simplify long-term expansion.
- Prioritizes **sensor integration, motion control, and autonomous behaviors**.
- Backed by **NASA and RRCC** for space-relevant robotics experimentation.

🛠 Hardware Setup (Core Build)

- **Microcontroller:** Arduino Mega 2560
- **Secondary Controller:** Raspberry Pi 4B running Ubuntu 22.04 LTS (used for higher-level tasks)
- **Mobility System:**
  - 4-wheel chassis (2-wheel drive)
  - 2 DC Motors connected via **L298N Motor Driver**
- **Sensor Suite:**
  - **Ultrasonic Sensors (x4):** Front-Left, Front-Center, Front-Right, Rear
  - **LIDAR Sensor:** Front distance scanning
  - **IMU (e.g., MPU6050):** For orientation control and terrain traversal adjustments
  - **Wheel Encoders (e.g., SPI-based):** GM3506 gimbal motors with positional feedback
- **Data Storage:** SD Card module (64GB capacity) for sensor logging and debugging

- **Wireless Communication:** Radio module (e.g., NRF24L01 or LoRa depending on use case)
- **On/Off Switch** + **Battery Pack (12V)**

Connectivity & Energy/Data Flow

Power Flow:

- Battery → On/Off Switch → Arduino VIN + Motor Driver VM
- Arduino 5V pin → Sensors (ultrasonic, IMU, SD)
- All components tied to **common ground**

Data Flow:

- **Ultrasonic sensors:** Trigger/Echo via Digital I/O pins
- **IMU:** I2C (SDA/SCL to Pins 20/21)
- **LIDAR:** Serial1 (TX/RX on Mega)
- **Encoders:** SPI
- **SD Card:** SPI (MOSI/MISO/SCK + CS)
- **Radio module:** SPI or Serial (based on model)

Functional Goals

1. Drive forward, backward, left, right
2. Detect and drive around obstacles
3. Traverse small and large obstacles
4. Recover from tipping (“flip over and drive”)
5. Self-orient using IMU after traversal
6. Record telemetry to SD card
7. Send telemetry wirelessly to ground station

Code Structure (Arduino)

- **Modular C++ architecture:**
  - IMUControl.cpp/.h: Interfaces with IMU sensor, handles orientation data.
  - MotorControl.cpp/.h: Controls L298N motor driver logic (PWM, direction).
  - OrientationControl.cpp/.h: Interprets IMU and encoder data for stability.
  - UltrasonicSensor.cpp/.h: Abstracts ultrasonic distance calculations.
  - Encoder.h/.cpp: Reads rotation data from gimbal encoders.
  - MuadDib_Main-1.ino: Main logic loop managing sensor data, PID loops, and commands.
- **Key Features:**
  - Uses PID control (via PIDController.h) to stabilize and control speed/heading.
  - Encoders + IMU used to determine exact position and orientation.
  - Sensor-based conditional logic (e.g., if object detected at 20cm, stop/turn).
  - Serial telemetry for LIDAR and ultrasonic outputs.
  - Optional machine learning offloaded to Raspberry Pi for decision-making.